

Technical Research Hand-Off: Multi-Instance Banknote Detection and Identification Under Extreme Occlusion

State of the Art: Literature Review and Existing Frameworks

Developing a computer vision system capable of robustly localizing and identifying banknotes in cluttered, hand-held, or highly overlapped scenarios requires a synthesis of classical geometric verification and modern deep learning topologies.¹ Historically, currency recognition systems operated under highly constrained assumptions, requiring flat, fully visible bills captured against uniform backgrounds under controlled lighting.² Real-world environments—characterized by fanned arrangements, hand occlusions, physical wear, and non-uniform illumination—render these early approaches obsolete.¹

Classical Local Invariant Feature Pipelines

To handle occlusion, rotation, and scale variations without massive training datasets, early research focused on local keypoint detectors and descriptors, primarily Scale-Invariant Feature Transform (SIFT) and Speeded Up Robust Features (SURF).² Because banknotes are printed using intricate, high-contrast engraving processes, local patches retain distinctive high-entropy information even when the global structure is disrupted.²

In a typical keypoint-based pipeline, such as the component-based system proposed for visual impairment assistance, a query image is processed to extract local keypoints, which are then matched against pre-computed descriptors of reference regions from a ground-truth image gallery.² The spatial relationships of these matched features are clustered to detect banknote presence and filter out chaotic background clutter.²

To handle overlapping notes, researchers developed iterative homography estimation techniques.³ The basic procedure relies on calculating a 2D projective transformation matrix, or

homography H , relating the plane of the reference banknote to the plane of the test image using RANSAC³:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Once a homography is estimated with a sufficient number of inlier matches, the boundary of the detected note is projected, and the matched inliers are removed from the global pool.⁴ The matching algorithm is then executed iteratively on the remaining keypoint pool to detect

subsequent overlapping instances.⁴

To eliminate false positives, the projected contours are passed through post-processing shape analysis algorithms that validate metrics such as convexity, area, circularity, and aspect ratio against known physical banknote dimensions.⁴ For mobile deployment, systems like the Elder-Digman pipeline applied parallelized feature matching using ratio distance thresholds:

$$d(f_{ref}, f_{query_1}) < 2 \cdot d(f_{ref}, f_{query_2})$$

This method cleared identical matching duplicates and validated overlapping arrangements, resolving up to twelve non-overlapping and four overlapping identical notes.³

Break Points on Fanned Khmer Riel Notes

When applied to fanned, overlapping Cambodian Khmer Riel notes, classical keypoint pipelines fail due to three primary bottlenecks:

- **Homogeneous Pattern Clashing:** Fanned banknotes of the same denomination present highly repetitive texture patterns (such as repeating Naga heads or Angkor Wat columns).¹⁰ Descriptor matching algorithms (e.g., vl_ubcmatch) experience high cross-matching rates across different physical bills, causing RANSAC to fail or calculate degenerate homographies that span multiple notes.³
- **Planar Projection Violations:** Banknotes held in a fanned stack are physically curved and wrinkled by hand pressure, violating the rigid 2D planar assumption required for homography calculations.³
- **Edge-Device Latency:** The quadratic complexity of keypoint matching and iterative RANSAC calculations on dense high-resolution frames runs slowly on mobile CPUs, making real-time interactive feedback impossible.²

Modern Single-Stage and Deep Cascaded Detectors

To meet real-time processing constraints on mobile hardware, research has transitioned to single-stage object detection architectures, particularly the You Only Look Once (YOLO) family.¹ Rather than relying on computationally heavy region proposal networks, YOLO models frame localization and classification as a unified regression task, processing frames in a single forward pass.¹

Recent currency systems leverage YOLOv8, YOLOv9, and YOLOv11 for multi-currency banknote and coin detection under partial occlusion.¹ For example, the Egyptian currency recognition system uses YOLOv10's Generalized Efficient Layer Aggregation Network (GELAN) and NMS-free training to achieve a mean Average Precision (mAP@0.5) of 0.9934 with ultra-low latency.¹⁵

To capture localized features, custom YOLO variants integrate Squeeze-and-Excitation (SE) blocks or Vision Transformer (ViT) encoders.⁶ These attention modules focus feature extraction on high-contrast security holograms, portraits, watermarks, and serial numbers, enhancing performance under extreme occlusion.⁶

For mobile deployment, these networks are converted into highly optimized TensorFlow Lite

(TFLite) or ONNX runtimes.¹ These runtimes achieve inference speeds under 10ms on Snapdragon or Apple Silicon neural processing units (NPUs).⁶

Break Points on Fanned Khmer Riel Notes

Despite their high speed, single-stage object detectors struggle with overlapping arrangements:

- **Bounding Box Overlap and Class Confusion:** During training, highly overlapping banknotes result in rectangular bounding boxes that enclose substantial pixel areas of neighboring notes.²¹ The network struggles to associate specific visual cues with the correct bounding box, leading to label noise.²¹
- **Non-Maximum Suppression (NMS) Failures:** When banknotes are fanned tightly, their predicted bounding boxes share highly similar center coordinates and spatial extents.¹³ Standard NMS steps often suppress valid adjacent notes as duplicate proposals, causing the system to detect only the topmost bill in the stack.¹

Amodal Instance Segmentation Topologies

Amodal instance segmentation addresses the limitations of standard object detectors by segmenting both the visible and occluded portions of an object.²² In a fanned stack of banknotes, an amodal model predicts the visible region (modal mask) and reconstructs the complete physical shape of the hidden notes (amodal mask).²²

Advanced frameworks, such as the Trilayer Convolutional Network (TCNet) and Bilayer Mask Predictor (BCNet), utilize Graph Convolutional Networks (GCN) to capture non-local pixel relationships and decouple overlapping objects into distinct structural layers.²⁷ More recent zero-shot methods, such as SAMEO, adapt the Segment Anything Model (SAM) as a flexible mask decoder.²³

SAMEO interfaces with diverse pre-trained front-end detectors to predict amodal masks for partially occluded objects, trained on large-scale synthetic datasets like Amodal-LVIS.²³ For bounding-box-only frameworks, amodal completion networks regress the full extent

coordinates $b^* = (x^*, y^*, w^*, h^*)$ from a noisy visible box $b = (x, y, w, h)$ by learning common occlusion and truncation patterns from the cropped visual features.²⁸





Break Points on Fanned Khmer Riel Notes

- While theoretically elegant, amodal segmentation topologies are difficult to deploy in production:
- **High Computational Latency on Edge Hardware:** Topologies like TCNet or SAM-based decoders contain non-standard operations (such as dynamic graph building, non-local attention, or high-dimensional transformer blocks) that lack optimized kernels in mobile deployment runtimes (TFLite, CoreML, NCNN).²⁹ This limits their use to server-side environments.²⁹
 - **Forced Completion Hallucinations:** When a banknote is heavily occluded (e.g., only a tiny corner is visible), amodal networks tend to hallucinate a complete rectangular note.²² If the visible corner is uninformative, the model guesses the underlying denomination based on learned shape priors, leading to high false-positive rates.²⁹

Synthesis of Recurring Engineering Patterns

Across the strongest vision systems analyzed, several recurring design principles emerge that directly address the challenge of identifying partially visible objects.

Localized Salient Feature Focus

Rather than evaluating banknotes as monolithic rectangles, successful architectures focus on localized, high-entropy features.² Under hand occlusions or fanned stacking, the global aspect ratio and context of the banknote are lost, but specific regional indicators remain intact.² These localized markers include numeric denomination prints, royal portraits, historical monuments, holographic strips, watermarks, and metallic security threads.¹⁷ By designing systems that learn these localized regions as sub-classes or utilizing neural attention mechanisms (such as Squeeze-and-Excitation blocks or Vision Transformers) that focus on these key regions, models maintain high classification accuracy even when only a small portion of the bill is exposed.⁶

Comprehensive Occlusion-Centric Data Synthesis

A key failure mode of naive deep learning models in production is their inability to generalize to occlusion patterns that were not represented in the training set.²¹ State-of-the-art models overcome this by incorporating synthetic data generation techniques.²³ Standard geometric and photometric augmentations (such as random cropping, perspective warping, brightness shifting, and blurring) are combined with advanced compositing strategies.¹ Specifically, synthetic fanned stacks are generated programmatically by layering alpha-masked banknote templates using random rotations and translations.²² Random hand-occluding masks are overlayed onto the banknote stacks to simulate physical handling.¹ This explicit exposure to layered, partially hidden objects teaches the model to focus on exposed visual slivers rather than relying on the presence of a complete rectangular boundary.²³

Explicit Confidence Calibration and Out-of-Distribution Rejection

In real-world deployment, misclassifying a banknote's denomination is significantly worse than outputting an "unknown" or low-confidence response.³⁶ Standard softmax layers tend to output overconfident predictions even for highly ambiguous, out-of-distribution, or heavily occluded inputs.³⁶ Robust systems counter this by implementing explicit rejection mechanisms.³⁶ These include configuring open-world detection heads (such as YOLO-UniOW) that leverage wildcard learning and text-image alignment in CLIP latent spaces to classify unrecognized regions as "unknown".³⁷ Alternatively, multi-stage pipelines apply secondary validation networks (such as Siamese verification models or lightweight patch classifiers) to cross-reference the primary detector's proposals, ensuring that a predicted denomination is backed by clear local evidence before displaying it to the user.⁴

Temporal Aggregation on Mobile Video Streams

Single-frame inference on mobile devices is highly susceptible to transient noise, motion blur, and momentary reflections.³ To stabilize predictions in production, applications run continuous inference over video frames and apply temporal smoothing.⁴⁰

Rather than relying on a single capture, the system maintains a running queue of predictions and aggregates class probabilities across consecutive frames using mechanisms like sliding-window average pooling or Kalman-filtered box tracking.³ This prevents momentary misclassifications and ensures that a denomination is only verified once its confidence score remains consistently high over a predefined temporal window.

Comparative Evaluation of Approach Families

A comprehensive evaluation of candidate architectures must balance localization capability, occlusion handling, mobile resource footprints, and training complexity.

YOLO-Style Object Detectors

YOLO architectures process input frames in a single pass, utilizing Path Aggregation Networks (PAN) and anchor-free heads to output spatial bounding boxes.¹

- **Localization and Separation:** Poor in dense stacks.¹³ When banknotes are fanned out,

their bounding boxes overlap heavily.²¹ Standard NMS suppresses valid adjacent notes, often resolving only the topmost note in a fan.¹³

- **Occlusion Performance:** Moderate.¹ If a key denomination cue (e.g., the portrait) is occluded, the box regression still converges based on the remaining borders, but the class prediction score drops rapidly.²¹
- **Rejection Capability:** Poor.³⁶ Standard YOLO heads use softmax classification, which outputs overconfident false predictions when exposed to ambiguous, cropped, or out-of-distribution inputs.³⁶
- **Mobile Suitability and Export Options:** Excellent.¹ YOLO models compile natively to TFLite, CoreML, ONNX, and NCNN.¹ An INT8 quantized model occupies 12–25MB and runs in under 8ms on standard mobile NPUs.⁶
- **Validation Requirements:** Test with overlapping arrangements of identical and mixed denominations, varying the overlap ratio from 10% to 90% to identify the exact NMS suppression thresholds.³

YOLO Segmentation Models

YOLO-seg architectures append a prototype mask generation branch to the backbone, producing pixel-level instance segmentation masks.²¹

- **Localization and Separation:** Excellent.²¹ By predicting exact boundary shapes, YOLO-seg isolates the visible portion of each banknote, preventing pixel leakage from adjacent notes during training.²¹
- **Occlusion Performance:** High.²¹ Can segment thin diagonal strips or corners.²¹ However, it is vulnerable to "white-on-white" boundary bleeding when overlapping banknotes share identical colors and textures.²¹
- **Rejection Capability:** Moderate.²¹ The mask shape acts as a physical validation metric; predicted masks with highly distorted geometries or extremely small surface areas can be programmatically rejected.⁴
- **Mobile Suitability and Export Options:** High.²⁰ Supported by ONNX Runtime Web and TFLite.³⁰ An INT8 quantized model occupies 20–35MB and runs at 15–30 FPS on WebGPU-enabled mobile browsers.²⁰
- **Validation Requirements:** Test the segmentation consistency under variable light and shadow gradients, ensuring that shadows cast by occluding hands do not cause a single banknote mask to split into separate fragments.¹

DETR / RT-DETR Topologies

Real-Time Detection Transformers (RT-DETR) replace anchor-based regressions with a multi-scale transformer encoder and decoder, utilizing bipartite matching to predict object queries without NMS post-processing.¹⁵

- **Localization and Separation:** High.¹⁵ By treating detection as a set prediction problem, RT-DETR avoids NMS suppression, allowing the model to detect heavily overlapping

objects with overlapping coordinates.¹⁵

- **Occlusion Performance:** High.⁴⁶ Self-attention layers capture global contextual dependencies, allowing the network to infer banknote presence from small exposed regions.²²
- **Rejection Capability:** Moderate.³⁶ Standard implementations still experience softmax overconfidence but can be modified using query-rejection thresholds.³²
- **Mobile Suitability and Export Options:** Moderate.²⁰ Can be exported to ONNX and TFLite, but transformer attention blocks run with higher latency on mobile NPUs compared to pure convolutional backbones.²⁰
- **Validation Requirements:** Measure latency on older, non-NPU mobile chipsets to verify that the self-attention operators do not trigger thermal throttling during continuous video streams.²⁰

Mask R-CNN / Two-Stage Segmenters

A traditional two-stage segmenter that uses a Region Proposal Network (RPN) to generate regions of interest (Rois), which are then classified and segmented by parallel heads.⁶

- **Localization and Separation:** Moderate to High.⁶ RPN proposals struggle to separate highly overlapping, homogeneous banknotes because the anchor boxes merge.¹³
- **Occlusion Performance:** High.²⁶ Standard two-stage systems can be extended with bilayer heads (like BCNet) to model occluder-occludee relationships.²⁷
- **Rejection Capability:** High.³⁶ The class probability is computed over highly refined RoI feature maps, reducing background noise and improving rejection metrics.³⁶
- **Mobile Suitability and Export Options:** Poor.¹³ Mask R-CNN is too heavy for real-time mobile browser use, with inference latencies often exceeding 150ms on mobile CPUs.¹³
- **Validation Requirements:** Benchmark model size and load times; a standard FP16 model exceeds 100MB, which can cause browser page crashes on resource-constrained devices.²⁰

Two-Stage Detector + Patch Classifier Pipelines

A hybrid pipeline where a lightweight, class-agnostic detector proposes candidate banknote crops, which are normalized and passed to a specialized patch classifier.⁶

- **Localization and Separation:** Moderate.⁶ Stage 1 is trained solely to locate "any paper banknote region," simplifying the localization task.³⁹ However, fanned notes are still subject to overlapping box limitations.¹³
- **Occlusion Performance:** High.¹⁷ Stage 2 can be trained specifically on small banknote crops (such as individual corners or portraits), making it highly robust to partial occlusions.²
- **Rejection Capability:** Outstanding.⁴ Stage 2 can be configured with a dedicated "unknown/rejection" class and calibrated using Platt scaling.⁴ If a cropped region lacks high-entropy denomination details, Stage 2 rejects it.¹⁷

- **Mobile Suitability and Export Options:** High.¹ Both models can be exported to TFLite or ONNX.⁶ Sequential execution runs efficiently within a 15–30ms envelope on mobile hardware.¹⁷
- **Validation Requirements:** Test for cumulative error propagation; verify that false region proposals from Stage 1 (such as rectangular books or papers) are successfully rejected by Stage 2.⁴

Classical Keypoint / Template Matchers

Non-neural frameworks that extract invariant local features (SIFT/SURF) and compute homography projections against a reference gallery.²

- **Localization and Separation:** High.³ Iterative homography calculation allows precise coordinate boundary localization and physical counting.⁴
- **Occlusion Performance:** High.³ Can identify banknotes with up to 75% occlusion, provided the visible portion contains a high density of keypoints.³
- **Rejection Capability:** High.³ If the number of inlier matches falls below an empirical threshold, the system rejects the input.³
- **Mobile Suitability and Export Options:** Moderate.³ Requires compiling C++ keypoint engines to WebAssembly (WASM).¹⁸ CPU execution causes high battery drain and thermal throttling under continuous frame loops.³
- **Validation Requirements:** Test with wrinkled, soiled, and dirty banknotes; wear and tear significantly reduces the number of stable keypoints, which can cause matching failures.²

Quantitative Architecture Evaluation Matrix

The following table summarizes the structural tradeoffs of each candidate approach family:

Approach Family	Labeling Format Required	Primary Strengths	Primary Weaknesses	Latency Profile (Mobile CPU / GPU)	Rejection Capability	Target Deployment Compatibility
YOLO Detectors ¹	2D Bounding Boxes	Fast inference, simple training pipeline. ¹	High box overlap, NMS suppresses adjacent notes. ¹³	15ms (CPU) / 4ms (GPU) ¹	Poor (softmax overconfidence) ³⁶	TFLite, CoreML, ONNX, NCNN ¹
YOLO	Polygonal	Precise	Vulnerabl	45ms	Moderate	ONNX

Segmenters ²¹	Instance Masks	boundary definition, resolves fanning. ²¹	e to white-on-white boundary bleeding. ²¹	(CPU) / 12ms (GPU) ²⁰	(geometric mask metrics) ⁴	Runtime Web, TFLite (WebGPU) ²⁰
RT-DETR ¹⁵	2D Bounding Boxes	No NMS needed, resolves overlapping boxes. ¹⁵	Higher computational overhead on mobile CPUs. ²⁰	60ms (CPU) / 18ms (GPU) ²⁰	Moderate (query probability) ³²	ONNX, TFLite (Modern NPUs only) ²⁰
Mask R-CNN ⁶	Polygonal Instance Masks	Highly accurate masks, models occlusions. ²⁷	High latency, too heavy for browser runtimes. ¹³	180ms (CPU) / 50ms (GPU) ¹³	High (RoI-level validation) ³⁶	PyTorch Mobile, Server-side Fallback ²⁹
Two-Stage Pipeline ⁶	Bounding Boxes + Crop Labels	Decouples localization; highly specialized classifier. ⁶	Sequential processing increases overall latency. ¹³	35ms (CPU) / 10ms (GPU) ¹⁷	Outstanding (Platt-scaled rejection) ⁴	Multi-Session ONNX / TFLite ⁶
Keypoint Matchers ²	Reference Images Only	Zero training data needed, high spatial precision. ³	Fails on worn bills, high CPU usage. ²	120ms (CPU only) ³	High (homography inlier threshold) ³	WebAssembly (compiled OpenCV) ¹⁸

Reusable Training and Data Engineering Methodologies

Building a robust pipeline for fanned Cambodian Khmer Riel banknotes requires structured data engineering and augmentation strategies.¹²

Labeling Paradigm: Modal vs. Amodal

When training models to detect fanned banknotes, labeling guidelines must be mathematically precise:

- **Modal Masks (Visible Regions Only):** Prioritize modal polygonal segmentation masks.²¹ When fanned banknotes overlap, they create a "white-on-white" occlusion problem (particularly for similarly colored denominations).²¹ If bounding boxes are used, the overlapping regions introduce severe training noise, as a box containing one note will enclose massive portions of neighboring notes.²¹ Modal masks isolate the exact visual evidence belonging to each specific instance, preventing the network from associating background patterns or adjacent bill details with the target denomination class.²¹
- **Amodal Masks (Full Estimated Extent):** Amodal masks are highly valuable for physical counting but should not be used as the primary feature input for denomination classification.²² Amodal predictions force the model to complete hidden geometries, which can cause denomination hallucination when only a tiny, uninformative corner of a bill is visible.²²

Labeling Rules for Small Visible slivers

To prevent the model from learning to guess denominations under extreme occlusion, enforce the following labeling rules:

- **Visible Area $> 30\%$** : Annotate as the specific denomination class. The note contains enough unique visual entropy (e.g., portions of the portrait or large denomination text) for reliable classification.³³
- **Visible Area between 15% and 30%** : Annotate as the class `unknown_banknote`. This trains the model to recognize the physical presence of a banknote (which is critical for counting) while explicitly preventing it from guessing a denomination based on insufficient visual evidence.³⁶
- **Visible Area $< 15\%$** : Ignore completely during annotation, or label as background.³⁶ Tiny exposed slivers do not contain stable patterns, and forcing the model to detect them will cause high false-positive rates and boundary hallucinations in production.³⁶

Measuring Partial-Visibility Difficulty

To maintain a balanced training set and prevent bias toward clean, unoccluded notes, compute a visibility ratio (V) for every training instance:

$$V = \frac{\text{Area}(M_{\text{modal}})}{\text{Area}(M_{\text{amodal}})}$$

This metric is calculated automatically during synthetic generation.²² Real-world validation images can be categorized by annotators into three difficulty tiers:

- **Easy** ($V \geq 0.7$): Standard foreground banknotes with minimal hand occlusion.
- **Medium** ($0.3 \leq V < 0.7$): Standard fanned arrangements, minor hand occlusions.
- **Hard** ($V < 0.3$): Deeply stacked notes where only thin borders or corner strips are exposed.

Programmatic Synthetic Generation Engine

To generate a diverse training set without manual capture bottlenecks, construct an offline synthetic pipeline that models the geometry of fanned playing cards and banknotes.²³ The generation engine should follow these steps:

1. **Source Asset Acquisition:** Capture high-resolution, flatbed scans of the front and back of all target KHR denominations.³
2. **Polygonal Alpha Masking:** Manually extract the exact boundary of each note to create a clean, transparent PNG template.²³
3. **Geometric Modeling of Card Fanning:** Model the spatial layout of fanned notes held in a hand.⁵¹ Banknotes held in a hand are typically fanned along a circular arc with a tight

pivot point.⁵¹ The transformation for each note k in a fan of N notes can be represented by a rotation θ_k and translation T_k relative to a common pivot point (x_p, y_p) :

$$\theta_k = \theta_{\text{start}} + k \cdot \frac{\Delta\theta}{N}$$

$$T_k = \begin{bmatrix} R \cos(\theta_k) \\ R \sin(\theta_k) \end{bmatrix}$$

4. **Compositing and Layering:** Render the banknotes sequentially from bottom to top.²⁷ For each layer, apply slight random perspective transformations to simulate the natural curling of paper, along with randomized coordinate translations and rotation adjustments.¹
5. **Occlusion Overlay:** Layer random high-resolution masks of human hands, fingers, and background clutter (tables, wallets, cash registers) onto the stack.¹
6. **Polygonal Mask Generation:** Because the rendering engine has full coordinate

knowledge of each layer, automatically compute the exact visible (modal) polygon for each banknote by subtracting the masks of all overlapping upper layers.²²

7. **Photometric Augmentations:** Apply global and localized lighting variations, glare, shadows, and camera noise models to match edge-device image sensors.¹



Essential Augmentations

To ensure the model generalizes to dirty, worn, and poorly lit KHR notes in the field, apply a specialized augmentation pipeline during training:

- **Physical Wear Simulation:** Apply random wrinkle displacement maps and local blurring to simulate the physical degradation of heavily circulated bills.²
- **Specular Glare and Low Light:** Add random high-intensity white Gaussian highlights to simulate reflections on plastic banknotes or under bright store lights, combined with low-contrast illumination drops to simulate night market environments.¹
- **Hand and Edge Cropping:** Synthesize random hand-occlusion masks overlapping the banknote edges, and randomly crop the fanned stack at the image borders to train the model on incomplete boundary shapes.¹

Active Learning and Hard Example Mining

To iteratively improve the model post-deployment, configure an active learning pipeline⁴⁷:

- In production, capture and flag frames where the local model detects a banknote but

outputs a low confidence score ($0.30 < C < 0.60$).³⁶

- Upload these flagged crops to a secure cloud-based labeling pool.³¹
- Annotate these hard examples with exact modal polygons and feed them back into the synthetic compositing engine to generate highly targeted training sets.²³

Production Pipeline Architectures

Designing a production-ready pipeline requires balancing on-device processing speed, network dependency, and accuracy constraints.

Architecture 1: Unified Single-Stage YOLOv11-seg (On-Device WebGPU)

A single-stage instance segmentation model running entirely within the client-side browser or mobile app.¹

- **Mechanism:** A lightweight YOLOv11n-seg model receives a 640x640 frame and directly outputs bounding boxes, class labels, and modal instance polygons in a single forward pass.¹
- **Evidence:** YOLO-based networks show high suitability for real-time mobile deployment under partial occlusion.¹
- **Data Requirements:** 20,000+ images of KHR banknote stacks annotated with exact modal polygons.²¹
- **Failure Modes:** Vulnerable to "white-on-white" boundary bleeding in highly compressed, homogeneous fanned stacks (e.g., several overlapping 10,000 Riel notes of identical color), causing masks to merge.²¹
- **Fast Prototyping:** Train a standard YOLOv11n-seg model on a synthetic dataset of 1,000 fanned KHR notes containing the three most common denominations (100, 1,000, and 10,000 Riel).⁴¹ Export to ONNX format and run real-time camera inference in a browser using onnxruntime-web with WebGPU acceleration.⁴⁰

Architecture 2: Class-Agnostic Detector + Specialized Crop Classifier (Two-Stage)

A sequential pipeline that isolates spatial localization from denomination classification.⁶

- **Mechanism:** Stage 1 uses an ultra-lightweight, class-agnostic YOLOv11-Nano detector trained to locate a single target: banknote_surface.¹ Stage 1 proposes bounding boxes around any visible paper banknote region.³⁹ The system crops these regions, normalizes their orientation, and passes them to Stage 2: a highly compact MobileNetV2 crop classifier trained to identify the denomination or output "unknown".⁶
- **Evidence:** Two-stage pipelines achieve high performance by focusing classification on local high-entropy regions.⁶
- **Data Requirements:** Stage 1 requires class-agnostic bounding boxes enclosing any visible banknote region.³⁹ Stage 2 requires a diverse dataset of cropped banknote regions

(corners, portraits, text regions) labeled by denomination, including a balanced "unknown" class composed of non-banknote paper crops and highly blurred images.⁶

- **Failure Modes:** If Stage 1 fails to detect a heavily occluded or folded banknote sliver, the note is never passed to Stage 2.¹³
- **Fast Prototyping:** Use a pre-trained class-agnostic detector as Stage 1. Train a lightweight MobileNetV2 classifier in PyTorch on a dataset of 5,000 cropped banknote regions.⁶ Convert both models to TFLite format and run sequential inference locally in a Python script.¹

Architecture 3: Multi-Cue System (Color + OCR + Local Templates)

A hand-crafted feature fusion pipeline that combines global color profiles, OCR detection, and local high-contrast templates.⁴

- **Mechanism:** A class-agnostic detector extracts banknote regions.³⁹ For each candidate crop, three parallel steps are executed:
 1. Calculate a global HSV color histogram to filter candidate denominations.²
 2. Run a fast local OCR scan (e.g., using a lightweight mobile OCR engine) to locate printed numeric values (such as "1000" or "᠑᠐᠐᠐").³³
 3. Extract SURF keypoints from specific high-contrast anchor regions (e.g., the King's portrait or temple monument) and match them against a reference database using homography validation.² The outputs are fused in a decision tree to verify the denomination.⁴
- **Evidence:** Prior banknote systems have combined visible colors with regional text printings to achieve robust classification.²
- **Data Requirements:** Coordinate definitions of high-contrast reference templates for each KHR denomination, along with a small validation set of 500 images containing diverse fanned configurations.³
- **Failure Modes:** Fails completely under very poor lighting or severe motion blur, where local templates and printed text are obscured.³
- **Fast Prototyping:** Build a Python prototype using OpenCV.¹⁸ Capture real-time frames, apply bilateral filtering to reduce noise, use classical contour detection to extract candidate regions, and run template matching on key visual regions.³

Architecture 4: Local Client-Side Screening + Cloud-Based ViT

Fallback

A distributed model that balances speed and accuracy.¹

- **Mechanism:** A lightweight TFLite model running locally on the user's smartphone performs real-time detection.¹ If the local model detects a banknote but the classification confidence falls within an ambiguous range (e.g., $0.35 < C < 0.65$), the high-resolution crop of that region is asynchronously transmitted to a cloud server.¹⁷ The server runs a heavy Vision Transformer segmentation model (such as SAM-2 or ViT-L) to

resolve the denomination under occlusion.¹⁷

- **Why it works:** Ensures absolute reliability for difficult edge cases (e.g., dirty, heavily wrinkled, or dark notes) while maintaining high-speed on-device feedback for clean, clear bills.¹
- **Required Data:** Identical to Architecture 1, plus a high-resolution dataset of noisy, blurred, and extreme edge-case banknotes to train the server-side ViT model.¹⁷
- **Failure Modes:** High network latency in areas with poor internet connectivity, causing the system to hang or display delayed results.¹⁸
- **Fast Prototyping:** Build a local client-side page using onnxruntime-web.⁵⁹ If local classification confidence is low, trigger an HTTP POST request sending the image crop to a simple Python-based Flask server running a PyTorch ViT model.¹⁷

Strategic Implementation Roadmap

To build a robust system for fanned Cambodian Khmer Riel banknotes, execute a two-phase development plan.

Phase 1: Two-Week Rapid Prototyping Agenda

The objective of Phase 1 is to build a minimal viable pipeline to validate edge-device execution speed and model inference under occlusion.

Days 1–3: Asset Capture and Pipeline Setup

- Obtain clean physical specimens of five standard KHR denominations: 100, 500, 1,000, 5,000, and 10,000 Riel.¹²
- Capture high-resolution, front-and-back scans of these bills using flatbed scanners to eliminate shadows and perspective warp.³
- Implement the programmatic synthetic generation pipeline in Python using OpenCV.¹⁸ Develop a script that generates 2,000 synthetic images of fanned, overlapping notes using the geometric fanning equations outlined in Section 4.²³ Configure the engine to output both YOLO bounding boxes and COCO-format polygon coordinates.²¹

Days 4–7: Training and Local Validation

- Train two model variants using the synthetic dataset:
 1. A lightweight bounding-box detector: **YOLOv11n**.¹
 2. A lightweight instance segmenter: **YOLOv11n-seg**.⁴¹
- Evaluate both models on a manually captured validation set of 100 real-world images featuring fanned Riels held in hand under diverse lighting conditions.¹
- Measure key performance indicators: Mean Average Precision at IoU=0.5 (mAP@50), boundary recall in dense stacks, and classification accuracy under hand occlusions.⁶

Days 8–10: On-Device Optimization and Export

- Export the trained PyTorch models to ONNX format, ensuring configuration with a fixed input shape of 640x640 and opset=12 to guarantee WebGPU compatibility.⁴²

- Apply 8-bit integer post-training quantization (INT8) to reduce model sizes to the target <25MB range.¹⁷
- Integrate the models into a simple client-side static web application utilizing the onnxruntime-web execution provider.⁴²

Days 11–14: Edge Performance Testing

- Deploy the static page and run real-time camera inference tests on target hardware (e.g., iPhone 13, middle-tier Android devices, standard laptop browser).¹
- Document processing speed (latency per frame in milliseconds) and identify any runtime issues.²⁰ Specifically, verify whether dynamic batch sizes over WebGPU trigger output zeroing errors, and lock batch size to 1 if necessary.⁴⁵

Phase 2: Long-Term Production Integration Roadmap

Following a successful prototype, transition the project to production over a 14-week cycle.



Unresolved Technical Risks and Open Questions

Before full-scale production deployment, several technical risks must be actively researched and resolved:

- **WebGPU Dynamic Batching Bottlenecks:** Standard ONNX-to-WebGPU conversion layers often suffer from compilation failures when handling dynamic batch sizes, which are necessary when a frame contains a variable number of detected banknote crops.⁴⁵ If multiple crops must be classified sequentially, this creates a major latency bottleneck on lower-end mobile devices.¹³
- **Homogeneous Pattern Bleeding:** When a stack consists entirely of clean, crisp notes of the same denomination (e.g., multiple blue 10,000 Riel notes), the visual texture is extremely uniform.²¹ Differentiating the boundary between the top note and the one beneath remains highly difficult for segmentation models, often resulting in merged masks and inaccurate note counts.²¹
- **Intense Environmental Interference:** High glare from plastic cash registers, extreme perspective distortion when looking down on a wallet, or low-light situations (e.g., night markets in rural Cambodia) can degrade edge detection.¹ Calibrating the classification confidence threshold to safely reject these frames as "unknown" without frustrating the user with excessive rejection prompts is a delicate UX balance.³⁶

Works cited

1. Real-Time Identification of Mixed and Partly Covered Foreign Currency Using YOLOv11 Object Detection - MDPI, accessed May 29, 2026, <https://www.mdpi.com/2673-2688/6/10/241>
2. Robust and Effective Component-based Banknote Recognition for the Blind - PMC - NIH, accessed May 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC3363999/>
3. Mobile Banknote Recognition and Conversion - Stacks are the Stanford, accessed May 29, 2026, https://stacks.stanford.edu/file/druid:yt916dh6570/Elder_Digman_Foreign_Bill_Recognition.pdf
4. Recognition of Banknotes in Multiple Perspectives Using Selective Feature Matching and Shape Analysis - inesc tec, accessed May 29, 2026, <https://repositorio.inesctec.pt/bitstream/123456789/4249/1/P-00M-9EY.pdf>
5. Forgery Detection and Value Identification of Euro Banknotes - PMC, accessed May 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC3649387/>
6. Real-Time Currency Detection and Voice Feedback for Visually Impaired Individuals - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2510.20267v1>
7. A SIFT-Based Feature Matching Algorithm Using Homography Estimation - Semantic Scholar, accessed May 29, 2026, <https://pdfs.semanticscholar.org/74eb/c41d816e2371a5ab4e1c1353a601dde6f4a7.pdf>
8. Currency-recognition by carlosmccosta, accessed May 29, 2026, <http://carlosmccosta.github.io/Currency-Recognition/>
9. Lecture 13 Homographies and RANSAC - mit csail, accessed May 29, 2026, <http://6.869.csail.mit.edu/fa12/lectures/lecture13ransac/lecture13ransac.pdf>
10. Cambodia Riel Banknotes royalty-free images - Shutterstock, accessed May 29,

- 2026, <https://www.shutterstock.com/search/cambodia-riel-banknotes>
11. Cambodian Riel (KHR) Currency Guide - Remitly, accessed May 29, 2026, <https://www.remitly.com/blog/en-ca/currency/cambodian-riel/>
 12. Cambodian riel - Wikipedia, accessed May 29, 2026, https://en.wikipedia.org/wiki/Cambodian_riel
 13. Machine learning object detector isn't as effective as I hoped it would be! - Hackaday.io, accessed May 29, 2026, <https://hackaday.io/project/27639/log/97907-machine-learning-object-detector-isn-t-as-effective-as-i-hoped-it-would-be>
 14. Fast SIFT Feature Matching Algorithm Based on Geometric Transformation - IEEE Xplore, accessed May 29, 2026, <https://ieeexplore.ieee.org/iel7/6287639/8948470/09075207.pdf>
 15. Recognizing Egyptian currency for people with visual impairment using deep learning models - PMC, accessed May 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12488862/>
 16. A Decade of You Only Look Once (YOLO) for Object Detection: A Review - IEEE Xplore, accessed May 29, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10820123/11237046.pdf>
 17. Fake Currency Detection Using Deep Learning - RJ Wave, accessed May 29, 2026, <https://rjwave.org/ijedr/papers/IJEDR2502037.pdf>
 18. Cash Eye - Currency Detection For Blind People Using Machine Learning - Zenodo, accessed May 29, 2026, <https://zenodo.org/records/19740528>
 19. DEVELOPMENT OF A MOBILE APP FOR NAIRA-CURRENCY IDENTIFICATION USING DEEP LEARNING CLASSIFICATION - Digital Library, accessed May 29, 2026, <https://library.ncs.org.ng/download/development-of-a-mobile-app-for-naira-currency-identification-using-deep-learning-classification/>
 20. Client-Side AI in 2025: What I Learned Running ML Models Entirely in the Browser - Medium, accessed May 29, 2026, <https://medium.com/@sauravgupta2800/client-side-ai-in-2025-what-i-learned-running-ml-models-entirely-in-the-browser-aa12683f457f>
 21. Playing Card Detection under Occlusion YOLO : r/computervision - Reddit, accessed May 29, 2026, https://www.reddit.com/r/computervision/comments/1r1i1i8/playing_card_detection_under_occlusion_yolo/
 22. Coarse-to-Fine Amodal Segmentation with Shape Prior - Amazon Science, accessed May 29, 2026, <https://assets.amazon.science/dc/d2/e393a6144eddaca3a971eb4a4e1c/coarse-to-fine-amodal-segmentation-with-shape-prior.pdf>
 23. [2503.06261] Segment Anything, Even Occluded - arXiv, accessed May 29, 2026, <https://arxiv.org/abs/2503.06261>
 24. Segment Anything, Even Occluded - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2503.06261v1>
 25. Partial modal completion under occlusion: What do modal and amodal percepts represent? | JOV | ARVO Journals, accessed May 29, 2026,

- <https://jov.arvojournals.org/article.aspx?articleid=2213216>
26. Segment Anything, Even Occluded [Quick Review] - Liner, accessed May 29, 2026, <https://liner.com/review/segment-everything-even-occluded>
 27. Amodal Instance Segmentation for Occlusion Handling with Balanced Overlapping Trilayer - Hyunjin Kim, accessed May 29, 2026, <https://kormachine.github.io/data/TCNet.pdf>
 28. Abhishek Kar - Amodal Completion and Size Constancy in Natural Scenes, accessed May 29, 2026, <https://abhishekkar.info/amodal.pdf>
 29. Seeking Advice: Mobile-Friendly Furniture Segmentation + Occlusion Handling - Models, accessed May 29, 2026, <https://discuss.huggingface.co/t/seeking-advice-mobile-friendly-furniture-segmentation-occlusion-handling/173738>
 30. Get started with LiteRT.js | Google AI Edge, accessed May 29, 2026, https://ai.google.dev/edge/litert/web/get_started
 31. How to add machine learning to your web application with ONNX Runtime, accessed May 29, 2026, <https://onnxruntime.ai/docs/tutorials/web/>
 32. Extreme Amodal Object Detector - Emergent Mind, accessed May 29, 2026, <https://www.emergentmind.com/topics/extreme-amodal-object-detector>
 33. Identification of Mixed Foreign Banknotes Using YOLOv8 Object Detection - ResearchGate, accessed May 29, 2026, https://www.researchgate.net/publication/393172464_Identification_of_Mixed_Foreign_Banknotes_Using_YOLOv8_Object_Detection
 34. Is it real? - Khmer Times, accessed May 29, 2026, <https://www.khmertimeskh.com/14567/is-it-real/>
 35. How to Spot Counterfeit Notes - Khmer Times, accessed May 29, 2026, <https://www.khmertimeskh.com/23690/how-to-spot-counterfeit-notes/>
 36. The Overlooked Elephant of Object Detection: Open Set, accessed May 29, 2026, https://openaccess.thecvf.com/content_WACV_2020/papers/Dhamija_The_Overlooked_Elephant_of_Object_Detection_Open_Set_WACV_2020_paper.pdf
 37. YOLO-UniOW: Efficient Universal Open-World Object Detection - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2412.20645v1>
 38. Best Generic Object Detection Models : r/computervision - Reddit, accessed May 29, 2026, https://www.reddit.com/r/computervision/comments/1jeg20i/best_generic_object_detection_models/
 39. How can vision AI models be used when playing cards - Ultralytics, accessed May 29, 2026, <https://www.ultralytics.com/blog/using-a-vision-ai-model-to-recognize-playing-cards>
 40. How to detect objects in videos in a web browser using YOLOv8 neural network and JavaScript - DEV Community, accessed May 29, 2026, <https://dev.to/andreygermanov/how-to-detect-objects-in-videos-in-a-web-browser-using-yolov8-neural-network-and-javascript-lfb>
 41. Comparing YOLOv11 and YOLOv8 for instance segmentation of occluded and non-occluded immature green fruits in complex orchard environment - arXiv,

- accessed May 29, 2026, <https://arxiv.org/html/2410.19869v3>
42. nomi30701/yolo-onnx-benchmark-web: yolo model benchmark onnxruntime webapp. Support WebGPU and wasm(cpu) - GitHub, accessed May 29, 2026, <https://github.com/nomi30701/yolo-model-benchmark-onnxruntime-web>
 43. Comparing YOLOv11 and YOLOv8 for instance segmentation of occluded and non-occluded immature green fruits in complex orchard environment - arXiv, accessed May 29, 2026, <https://arxiv.org/abs/2410.19869>
 44. Run AI Models in the Browser with WebGPU & WASM - Mad Devs, accessed May 29, 2026, <https://maddevs.io/writeups/running-ai-models-locally-in-the-browser/>
 45. [Performance] Running YOLOv8-seg.onnx with Dynamic Batch Size on GPU · Issue #21103 · microsoft/onnxruntime - GitHub, accessed May 29, 2026, <https://github.com/microsoft/onnxruntime/issues/21103>
 46. Tracking Any Object Amodally - arXiv, accessed May 29, 2026, <https://arxiv.org/html/2312.12433v2>
 47. (PDF) Active and Incremental Learning with Weak Supervision - ResearchGate, accessed May 29, 2026, https://www.researchgate.net/publication/338676363_Active_and_Incremental_Learning_with_Weak_Supervision
 48. Banknote recognition: investigating processing and cognition framework using competitive neural network - PMC, accessed May 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC5264750/>
 49. Banknote Recognition: Investigating Processing and Cognition Framework Using Competitive Neural Network - PubMed, accessed May 29, 2026, <https://pubmed.ncbi.nlm.nih.gov/28174613/>
 50. Cambodia Currency: Everything About Cambodian Riels 2026, accessed May 29, 2026, <https://cambodiatravel.com/cambodia-currency/>
 51. Roulette Fanimation Playing Cards - 52Kards, accessed May 29, 2026, <https://shop.52kards.com/products/roulette-fanimation>
 52. Banknotes in Circulation, accessed May 29, 2026, https://www.nbc.gov.kh/english/about_the_bank/banknotes_in_circulation.php
 53. Cards Fanned royalty-free images - Shutterstock, accessed May 29, 2026, <https://www.shutterstock.com/search/cards-fanned>
 54. Card Fan Flourish Tutorial! - YouTube, accessed May 29, 2026, <https://www.youtube.com/shorts/6t1r7md4Xa8>
 55. How to Perfectly Fan and Spread Cards | Scam Nation - YouTube, accessed May 29, 2026, https://www.youtube.com/watch?v=ShM4_2YD6K4
 56. Active and Incremental Learning with Weak Supervision, accessed May 29, 2026, <https://d-nb.info/1208687417/34>
 57. AML Machine Learning: How It's Used to Detect Money Laundering - SEON, accessed May 29, 2026, <https://seon.io/resources/aml-machine-learning/>
 58. Cambodia Currency Project Object Detection Model by Khmer Riel Classification Computer Vision - Roboflow Universe, accessed May 29, 2026, <https://universe.roboflow.com/khmer-riel-classification-computer-vision/cambodia-currency-project>
 59. Classify images with ONNX Runtime and Next.js | onnxruntime, accessed May 29,

2026,

<https://onnxruntime.ai/docs/tutorials/web/classify-images-nextjs-github-template.html>

60. Newest 'onnxruntime' Questions - Stack Overflow, accessed May 29, 2026,
<https://stackoverflow.com/questions/tagged/onnxruntime?tab=Newest>